



TECHNOLOGY
SOLUTIONS UK LTD
part of **HID**

ANDROID™ SDK V2.4: GETTING STARTED

CONTENT

Getting Started.....	3
A note about ePop-Loq® readers.....	3
Using the Android™ v2.4 SDK.....	4
Converting a Project from the <i>Bluetooth</i> ®-only SDK to the Unified v2.4 SDK.....	5
Adding Support for TSL® ASCII 2 Readers to a Project.....	6
Modify the build.Gradle file.....	6
Modify AndroidManifest.xml.....	6
Use a single instance of AsciiCommander.....	6
Modify Your App's Activity.....	7
Add a Select Reader screen.....	13
Converting a Project from the ePop-Loq® SDK to the Unified v2.4 SDK.....	16
Hints and Tips.....	16
About.....	19
About TSL®.....	19
Contact.....	19
About HID Global.....	19

GETTING STARTED

PLEASE NOTE: This document is provided in two versions - a .PDF and a plain .TXT file. We recommend copying and pasting code from the plain .TXT file only.

A detailed description of the SDK contents can be found in the included Developer Guide:

Android SDK v2.x Developer Guide.pdf

Nine Sample Projects are included in the SDK:

Inventory -	How to scan for tags and receive barcode data
ReadWrite -	How to Access and modify transponder memory banks.
Trigger -	How to change the operation of the trigger and receive trigger notifications
LicenceKey -	How to read and write a licence key to the reader
ConfigureBarcode -	How to change the barcode engine's active symbologies
ReaderManager -	How to connect to a Reader - automatically (for ePop-Loq®/USB) or using the DeviceListActivity
TagFinder -	How to configure the Reader to provide signal strength (RSSI) values for a specified tag
BlockPermalock -	How to use block-level locking (requires Tag support)
HFLFSupport -	How to control the 2173 HF/LF Reader using ASCII 2 commands or the SimpleProtocol*

These projects contain examples of how to use the most common commands of the Ascii 2 Protocol and are provided with gradle files ready to be opened in Android Studio v2.3.3 or greater.

* An ASCII pass-through command is used to control the HF/LF module in the reader, providing access to the module's full capabilities. See document: Pass Through mode for the 2173 HF/LF reader

<https://www.tsl.com/download-manager/pass-through-mode/>

A note about ePop-Loq® readers

The ePop-Loq® connector has two modes of operation:

- Charge Only: No data communications occur to the mounted terminal
- Charge & Data: The terminal will have a data connection to the Reader

The ePop-Loq® mode can be changed using the TSL Reader Configuration App v1.1 or greater.

All TSL ePop-Loq Readers are shipped in the "Charge & Data" mode, by default.

Note: when placed in the Docking cradle, in either mode, there will be no data connection through the ePop-Loq® connector but the Reader and mounted terminal will be charged. USB communications can be achieved with the docked Reader by using the Data port on the Docking cradle and an OTG USB cable to an (OTG capable) Android device.

Using the Android™ v2.4 SDK

The Android SDK v2.4 is designed to provide maximal backwards-compatibility for developers converting projects using the v1.x *Bluetooth*®-only Android SDK.

To add the Rfid.AsciiProtocol-Library to your Android Studio project refer to these instructions:

Using the Rfid.AsciiProtocol-Library with Android Studio.txt

When developing with ePop-Loq® and USB OTG devices you will probably want to use wireless debugging:

<https://developer.android.com/studio/command-line/adb.html#wireless>

If you are working with a project that has not used a TSL® Android SDK before then skip to the section “Adding Support for TSL® ASCII 2 Readers to a Project” on page 6.

Alternatively, you may wish to follow the step-by-step tutorial “Creating the ReaderManager Sample project” which will guide you through the complete process of creating a new Android Studio Project that works with TSL ASCII 2 Readers.

CONVERTING A PROJECT FROM THE *BLUETOOTH*[®]-ONLY SDK TO THE UNIFIED V2.4 SDK

If you wish to convert an App that uses the *Bluetooth*[®] SDK (v1.4.7 or earlier) to an App that also supports directly connected ePop-Loq[®]/USB readers then follow the steps below. Some or all of the steps may apply depending on how closely your App is modelled on the *Bluetooth*[®] SDK Sample projects.

These steps outline how to remove the redundant *Bluetooth*[®]-based code. Support for ePop-Loq[®]/USB Readers can then be added using the information in the section “Adding Support for TSL[®] ASCII 2 Readers to a Project” on page 6.

- Remove **TSLBluetoothDeviceActivity** and any imports using it.
- Activities no longer need to extend **TSLBluetoothDeviceActivity** so use an appropriate alternative such as **AppCompatActivity**
- Remove **TSLBluetoothDeviceApplication** and the corresponding manifest entry:

```
android:name="com.uk.tsl.rfid.TSLDeviceApplication"
```

- Implement the convenience method for getting the AsciiCommander in your Activity (if needed):

```
/**
 * @return the current AsciiCommander
 */
protected AsciiCommander getCommander()
{
    return AsciiCommander.sharedInstance();
}
```

- **ConnectionState** has moved to `com.uk.tsl.rfid.asciiprotocol.device`. So, remove **AsciiCommander** class from lines such as **AsciiCommander.ConnectionState.CONNECTING** and then fix imports.
- **hasConnectedSuccessfully()** method is no longer available on **AsciiCommander**. The **Reader.wasLastConnectSuccessful()** method can be used to code a replacement within the App.

We have tried to ensure that App business logic, that interacts with Readers through **AsciiCommander**, should be minimally affected by v2.4 SDK changes.

Your App should now resemble an application that does not have any code to connect to a TSL[®] Reader.

The next section describes how to add in support for the new **ReaderManager** - additional information about converting existing projects is in the following sections.

Note that the occasional step below may not be needed when converting as code will already exist.

ADDING SUPPORT FOR TSL® ASCII 2 READERS TO A PROJECT

These steps demonstrate how to add support for TSL® ASCII 2 Readers to an existing App that does not currently use the TSL® Android SDK.

The Android SDK v2.4 supports both *Bluetooth®* and *ePop-Loq®* connected Readers.

The procedure below also shows how to:

- Use the DeviceListActivity project to allow the User to choose which Reader to use.
- Automatically connect to an attached ePop-Loq®/USB Reader.
- Disconnect from the current Reader when the App is paused - this allows multiple Apps to share a connection to the same Reader.

Modify the build.gradle file

Add the Rfid.AsciiProtocol.Library to the project dependencies:

```
implementation project(':Rfid.AsciiProtocol-Library')
```

Modify AndroidManifest.xml

Add the following to AndroidManifest.xml to allow the App to be launched when an ePop-Loq®/USB Reader connects - otherwise the User will be repeatedly presented with the Allow <App Name> to access..." dialog.

In **<Activity>**

```
<intent-filter>
<action android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED" />
</intent-filter>
<meta-data android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED"
android:resource="@xml/device_filter" />
```

Add this to the mainActivity intent to ensure only one instance of the activity is launched if the App is already open and a device is attached. Otherwise a new Activity is created for each time the device is connected.

```
android:launchMode="singleTask"
```

Use a single instance of AsciiCommander

We currently recommend using a single instance of AsciiCommander and to support this the AsciiCommander class can create and provide a shared instance.

The Activity can call the **AsciiCommander.createSharedInstance(Context)** method , in **onCreate()** , to ensure the shared instance exists.

Modify Your App's Activity

```
import com.uk.tsl.rfid.asciiprotocol.device.Reader;
```

Add variable for the currently connected reader - mReader :

```
// The Reader currently in use
private Reader mReader = null;
```

Also need to track when theDeviceList Activity is in use:

```
private boolean mIsSelectingReader = false;
```

The Sample projects add a convenience method to access the AsciiCommander and this is used in the code examples below:

```
/**
 * @return the current AsciiCommander
 */
protected AsciiCommander getCommander()
{
    return AsciiCommander.sharedInstance();
}
```

With the 'unified' *Bluetooth*® and ePop-Loq® SDK, *Bluetooth*® Readers will also appear in the ReaderManager's ReaderList. The sample code for the ePop-Loq® SDK ensured that *Bluetooth*®-only Readers in the list will be ignored.

To add support for *Bluetooth*® Readers and provide an interface for Users to select Readers follow the steps starting from the "Modify Your App's Activity" section above.

In **onCreate()**:

Configure the AsciiCommander and the ReaderManager:

```
// Ensure the shared instance of AsciiCommander exists
AsciiCommander.createSharedInstance(getApplicationContext());

final AsciiCommander commander = getCommander();

// Ensure that all existing responders are removed
commander.clearResponders();

// Add the LoggerResponder - this simply echoes all lines received from the reader to
the log
// and passes the line onto the next responder
// This is ADDED FIRST so that no other responder can consume received lines before they
are logged.
commander.addResponder(new LoggerResponder());

// Add responder to enable the synchronous commands
commander.addSynchronousResponder();

// Configure the ReaderManager when necessary
ReaderManager.create(getApplicationContext());
```

```
// Add observers for changes
ReaderManager.sharedInstance().getReaderList().readerAddedEvent().
addObserver(mAddedObserver);
ReaderManager.sharedInstance().getReaderList().readerUpdatedEvent().
addObserver(mUpdatedObserver);
ReaderManager.sharedInstance().getReaderList().readerRemovedEvent().
addObserver(mRemovedObserver);
```

We add observers to the ReaderManager's ReaderList so that we know when a Reader is attached or detached.

When the activity is destroyed we need to stop observing so, **onDestroy()** (may need to be created) should look like this:

```
@Override
protected void onDestroy()
{
    super.onDestroy();

    // Remove observers for changes
    ReaderManager.sharedInstance().getReaderList().readerAddedEvent().
removeObserver(mAddedObserver);
    ReaderManager.sharedInstance().getReaderList().readerUpdatedEvent().
removeObserver(mUpdatedObserver);
    ReaderManager.sharedInstance().getReaderList().readerRemovedEvent().
removeObserver(mRemovedObserver);
}
```

To allow other Apps to use the Reader we need to stop using it when the App is paused and reconnect when the App resumes. A slight complication here is that when an ePop-Loq® reader is connected Android will pause and resume the Activity - we use the ReaderManager to determine if this is the reason for the pause using the **ReaderManager.sharedInstance().didCauseOnPause()** method.

If the Activity is starting up then it will not know which Reader is in use. The code below will also find the (first) connected USB Reader and tell AsciiCommander to use that.

In **onResume()**, add after **super.onResume()**:

```
// Register to receive notifications from the AsciiCommander
LocalBroadcastManager.getInstance(this).registerReceiver(mMessageReceiver, new
IntentFilter(AsciiCommander.STATE_CHANGED_NOTIFICATION));

// Remember if the pause/resume was caused by ReaderManager - this will be cleared when
ReaderManager.onResume() is called
boolean readerManagerDidCauseOnPause = ReaderManager.sharedInstance().didCauseOnPause();

// The ReaderManager needs to know about Activity lifecycle changes
ReaderManager.sharedInstance().onResume();

// The Activity may start with a reader already connected (perhaps by another App)
// Update the ReaderList which will add any unknown reader, firing events appropriately
ReaderManager.sharedInstance().updateList();

// Locate a Reader to use when necessary
AutoSelectReader(!readerManagerDidCauseOnPause);

mIsSelectingReader = false;
```


In `onPause()`, add after `super.onPause()`:

```
// Unregister from notifications for the AsciiCommander
LocalBroadcastManager.getInstance(this).unregisterReceiver(mMessageReceiver);

// Disconnect from the reader to allow other Apps to use it
// unless pausing when USB device attached or using the DeviceListActivity to select a
Reader
if( !mIsSelectingReader && !ReaderManager.sharedInstance().didCauseOnPause() && mReader
!= null )
{
    mReader.disconnect();
}

ReaderManager.sharedInstance().onPause();
```

Implement the `AutoSelectReader()` method:

```
//
// Automatically select the Reader to use
//
private void AutoSelectReader(boolean attemptReconnect)
{
    ObservableReaderList readerList = ReaderManager.sharedInstance().getReaderList();
    Reader usbReader = null;
    if( readerList.list().size() >= 1)
    {
        // Currently only support a single USB connected device so we can safely take the
        // first CONNECTED reader if there is one
        for (Reader reader : readerList.list())
        {
            if (reader.hasTransportOfType(TransportType.USB))
            {
                usbReader = reader;
                break;
            }
        }
    }

    if( mReader == null )
    {
        if( usbReader != null )
        {
            // Use the Reader found, if any
            mReader = usbReader;
            getCommander().setReader(mReader);
        }
    }
    else
    {
        // If already connected to a Reader by anything other than USB then
        // switch to the USB Reader
        IAsciiTransport activeTransport = mReader.getActiveTransport();
        if ( activeTransport != null && activeTransport.type() != TransportType.USB &&
usbReader != null)
        {

```

```

        appendMessage("Disconnecting from: " + mReader.getDisplayName() + "\n");
        mReader.disconnect();

        mReader = usbReader;

        // Use the Reader found, if any
        getCommander().setReader(mReader);
    }
}

// Reconnect to the chosen Reader
if( mReader != null
    && !mReader.isConnecting()
    && (mReader.getActiveTransport() == null || mReader.getActiveTransport().
connectionStatus().value() == ConnectionState.DISCONNECTED))
{
    // Attempt to reconnect on the last used transport unless the ReaderManager is cause
of OnPause (USB device connecting)
    if( attemptReconnect )
    {
        if( mReader.allowMultipleTransports() || mReader.getLastTransportType() == null
)
        {
            // Reader allows multiple transports or has not yet been connected so
connect to it over any available transport
            if( mReader.connect() )
            {
                appendMessage("Connecting to: " + mReader.getDisplayName() + "\n");
            }
        }
        else
        {
            // Reader supports only a single active transport so connect to it over the
transport that was last in use
            if( mReader.connect(mReader.getLastTransportType()) )
            {
                appendMessage("Connecting (over last transport) to: " + mReader.
getDisplayName() + "\n");
            }
        }
    }
}
}
}

```

Add the ReaderList Observer methods - the removed event is needed to recognise when the removed reader is the one that is in use:

```

// ReaderList Observers
Observable.Observer<Reader> mAddedObserver = new Observable.Observer<Reader>()
{
    @Override
    public void update(Observable<? extends Reader> observable, Reader reader)
    {
        // See if this newly added Reader should be used
        AutoSelectReader(true);
    }
};

```

```

Observable.Observer<Reader> mUpdatedObserver = new Observable.Observer<Reader>()
{
    @Override
    public void update(Observable<? extends Reader> observable, Reader reader)
    {
    }
};

Observable.Observer<Reader> mRemovedObserver = new Observable.Observer<Reader>()
{
    @Override
    public void update(Observable<? extends Reader> observable, Reader reader)
    {
        // Was the current Reader removed
        if( reader == mReader)
        {
            mReader = null;

            // Stop using the old Reader
            getCommander().setReader(mReader);
        }
    }
};
    
```

These methods modify AsciiCommander's Reader property and in doing so cause AsciiCommander to raise notifications about the connection state.

Implement the AsciiCommander message receiver to track the changes of connection state:

```

//
// Handle the messages broadcast from the AsciiCommander
//
private BroadcastReceiver mMessageReceiver = new BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        String connectionStateMsg = getCommander().getConnectionState().toString();
        Log.d("", "AsciiCommander state changed - isConnected: " + getCommander().
isConnected() + " (" + connectionStateMsg + ")");

        if(getCommander() != null)
        {
            if (getCommander().isConnected())
            {
                // Report the battery level when Reader connects
                BatteryStatusCommand bCommand = BatteryStatusCommand.synchronousCommand();
                getCommander().executeCommand(bCommand);
                int batteryLevel = bCommand.getBatteryLevel();
            }
            else if(getCommander().getConnectionState() == ConnectionState.DISCONNECTED)
            {
                // A manual disconnect will have cleared mReader
                if( mReader != null )
                {
                    // See if this is from a failed connection attempt
                    if (!mReader.wasLastConnectSuccessful())
                    {
                    }
                }
            }
        }
    }
};
    
```

```

        // Unable to connect so have to choose reader again
        mReader = null;
    }
}
}
}
};

```

Your Activity can now detect when ePop-Loq® or USB Readers are connected and configure the **AsciiCommander.sharedInstance()** ready for use.

If you only wish to support ePop-Loq Readers then skip the following section otherwise follow the steps below to add the DeviceList Activity. This gives easy access to *Bluetooth*® Readers and allows the User to easily swap between Readers.

Alternatively, you may wish to follow the step-by-step tutorial “Creating the ReaderManager Sample project” which will guide you through the complete process of creating a new Android Studio Project that works with TSL ASCII 2 Readers.

Add a Select Reader screen

Add the DeviceList project to the target project:

- 1) Copy the DeviceList project source to the target project's root folder
- 2) In Settings.gradle add:

```
include ':DeviceList'
```

Add a dependency for the DeviceListActivity project to the build.gradle file for the project:

```
dependencies {

    implementation project(':DeviceList')
}
```

This Activity provides a Reader selection screen (with images and Reader descriptions) that can be used by the main Activity to select any of the available Readers.

To manage the current Reader, add two menu items to the Activity:

First add the Strings for the menu text in res/values/strings.xml:

```
<string name="connect_reader_menu_item_text">Connect Reader</string>
<string name="change_reader_menu_item_text">Change Reader</string>
<string name="disconnect_reader_menu_item_text">Disconnect Reader</string>
```

In **res/menu/** create a menu file e.g. reader_menu.xml, containing:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android" >

    <item android:id="@+id/connect_reader_menu_item" android:title="@string/connect_
reader_menu_item_text"/>
    <item android:id="@+id/disconnect_reader_menu_item" android:title="@string/
disconnect_reader_menu_item_text"/>

</menu>
```

In the **Activity.java**

Add fields for the menu items:

```
private MenuItem mConnectMenuItem;
private MenuItem mDisconnectMenuItem;
```

Implement or modify the **onCreateOptionsMenu()** method:

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    // Inflate the menu; this adds items to the action bar if it is present.
    getMenuInflater().inflate(R.menu.reader_menu, menu);

    mConnectMenuItem = menu.findItem(R.id.connect_reader_menu_item);
    mDisconnectMenuItem= menu.findItem(R.id.disconnect_reader_menu_item);

    return true;
}
```

In `onPrepareOptionsMenu()` enable/disable the menu appropriately:

```
/**
 * Prepare the menu options
 */
@Override
public boolean onPrepareOptionsMenu(Menu menu)
{
    boolean isConnected = getCommander().isConnected();
    mDisconnectMenuItem.setEnabled(isConnected);

    mConnectMenuItem.setEnabled(true);
    mConnectMenuItem.setTitle( (mReader != null && mReader.isConnected() ? R.string.change_
reader_menu_item_text : R.string.connect_reader_menu_item_text));

    return super.onPrepareOptionsMenu(menu);
}
```

In method `onOptionsItemSelected()` replace `if(){} for settings` with::

```
switch (id) {

    case R.id.connect_reader_menu_item:
        // Launch the DeviceListActivity to see available Readers
        mIsSelectingReader = true;
        int index = -1;
        if( mReader != null )
        {
            index = ReaderManager.sharedInstance().getReaderList().list().
indexOf(mReader);
        }
        Intent selectIntent = new Intent(this, DeviceListActivity.class);
        if( index >= 0 )
        {
            selectIntent.putExtra(EXTRA_DEVICE_INDEX, index);
        }
        startActivityForResult(selectIntent, DeviceListActivity.SELECT_DEVICE_
REQUEST);

        return true;

    case R.id.disconnect_reader_menu_item:
        if( mReader != null )
        {
            mReader.disconnect();
            mReader = null;
        }

        return true;
}
```

Add a handler for the result of the DeviceList Activity:

```
//
// Handle Intent results
//
public void onActivityResult(int requestCode, int resultCode, Intent data) {

    switch (requestCode)
```

```

    {
        case DeviceListActivity.SELECT_DEVICE_REQUEST:
            // When DeviceListActivity returns with a device to connect
            if (resultCode == Activity.RESULT_OK)
            {
                int readerIndex = data.getExtras().getInt(EXTRA_DEVICE_INDEX);
                Reader chosenReader = ReaderManager.sharedInstance().getReaderList().list().
get(readerIndex);

                int action = data.getExtras().getInt(EXTRA_DEVICE_ACTION);

                // If already connected to a different reader then disconnect it
                if( mReader != null )
                {
                    if( action == DeviceListActivity.DEVICE_CHANGE || action ==
DeviceListActivity.DEVICE_DISCONNECT)
                    {
                        mReader.disconnect();
                        if(action == DeviceListActivity.DEVICE_DISCONNECT)
                        {
                            mReader = null;
                        }
                    }
                }

                // Use the Reader found
                if( action == DeviceListActivity.DEVICE_CHANGE || action ==
DeviceListActivity.DEVICE_CONNECT)
                {
                    mReader = chosenReader;
                    getCommander().setReader(mReader);
                }
                break;
            }
    }
}

```

The Activity is now set up to monitor the arrival and disappearance of TSL® Readers and equipped with an AsciiCommander that can be used to issue commands and receive responses from the reader.

CONVERTING A PROJECT FROM THE EPOP-LOQ® SDK TO THE UNIFIED V2.4 SDK

The new SDK can be adopted with minimal changes if *Bluetooth®* Reader support is not required

- Delete the TSLDeviceApplication file.
- Remove from AndroidManifest.xml:

```
android:name="com.uk.tsl.rfid.TSLDeviceApplication"
```

- In **onCreate()** call the **createSharedInstance** method of **AsciiCommander** before it is used to ensure the instance exists.

```
AsciiCommander.createSharedInstance(getApplicationContext());
```

- In the Sample code projects: change the definition of **getCommander()**:

```
/**
 * @return the current AsciiCommander
 */
protected AsciiCommander getCommander()
{
    return AsciiCommander.sharedInstance();
}
```

With the 'unified' BT and ePop-Loq SDK, Bluetooth Readers will also appear in the ReaderManager's ReaderList.

The sample code for the ePop-Loq SDK ensured that Bluetooth-only Readers in the list will be ignored.

To add support for Bluetooth Readers and provide an interface for Users to select Readers follow the steps starting from the "Modify your App's Activity" section above.

Hints and Tips

Use the **AsciiCommander.getConnectionState()** to determine the detailed connection state of **AsciiCommander**. Alternatively, **AsciiCommander.isConnected()** method is often sufficient. These values reflect the states of **AsciiCommander**'s current Reader.

Once **AsciiCommander** is connected, e.g. **AsciiCommander.isConnected()** returns true, then the **AsciiCommander.getDeviceProperties()** method will return an instance that describes the currently connected Reader. This can be used to determine the correct range for antenna power e.g.

```
if(getCommander() != null && getCommander().isConnected())
{
    mMinimumPower = getCommander().getDeviceProperties().getMinimumCarrierPower();
    mMaximumPower = getCommander().getDeviceProperties().getMaximumCarrierPower();
}
else
{
    mMinimumPower = AntennaParameters.MinimumCarrierPower;
    mMaximumPower = AntennaParameters.MaximumCarrierPower;
}
```


For example, to execute a command and wait for the result (a synchronous command):

```
if(getCommander() != null && getCommander().isConnected())
{
    BatteryStatusCommand bCommand = BatteryStatusCommand.synchronousCommand();
    getCommander().executeCommand(bCommand);
    int batteryLevel = bCommand.getBatteryLevel();
}
```

See the Sample Projects for more examples, including the use of asynchronous commands.

To see the raw data coming from the Reader, in **onCreate()**, you can create a simple IAsciiResponder and add it as one of the first in the AsciiCommanders Responder chain as follows:

```
// Add a simple Responder that sends the Reader output to the App message list
commander.addResponder(new IAsciiCommandResponder() {
    @Override
    public boolean isResponseFinished() { return false; }

    @Override
    public void clearLastResponse() {}

    @Override
    public boolean processReceivedLine(String fullLine, boolean moreLinesAvailable)
    {
        appendMessage(">" + fullLine);
        return false; // don't consume the line - allow others to receive it
    }
});
```

If there is no convenient area for messages in your UI then the LoggerResponder will output all Reader responses to the standard log file.

In **onCreate()** add:

```
// Add the LoggerResponder - this simply echoes all lines received from the reader to
the log
// and passes the line onto the next responder
// This is ADDED FIRST so that no other responder can consume received lines before they
are logged.
commander.addResponder(new LoggerResponder());
```

If you need to connect/disconnect from the chosen Reader programmatically then use the following:

```
if( mReader != null ) mReader.connect();
if( mReader != null ) mReader.disconnect();
```

Additional useful information about the use of the API can be found in:

Application Note ASCII 2 API Overview.pdf

The latest reference to the ASCII 2 protocol is also included:

TSL_ASCII_Protocol_2.5_Rev_A.pdf

Finally, if you need any technical help using this SDK please contact support@tsl.com and we will do our best to resolve your issues.

ABOUT

About TSL®



Technology Solutions UK Ltd (TSL®), part of HID Global, is a leading manufacturer of high performance mobile RFID readers used to identify and track products, assets, data or personnel.

For over two decades, TSL® has delivered innovative data capture solutions to Fortune 500 companies around the world using a global network of distributors and system integrators. Specialist in-house teams design all aspects of the finished products and software ecosystems, including electronics, firmware, application development tools, RF design and injection mould tooling.

TSL® is an ISO 9001:2015 certified company.



ISO 9001: 2015

Contact

Address:	Technology Solutions (UK) Ltd, Suite A, Loughborough Technology Centre, Epinal Way, Loughborough, Leicestershire, LE11 3GE, United Kingdom.
Telephone:	+44 1509 238248
Fax:	+44 1509 214144
Email:	enquiries@tsl.com
Website:	www.tsl.com

About HID Global



HID Global powers the trusted identities of the world's people, places and things. We make it possible for people to transact safely, work productively and travel freely. Our trusted identity solutions give **people** convenient access to physical and digital **places** and connect **things** that can be identified, verified and tracked digitally. Millions of people around the world use HID products and services to navigate their everyday lives, and billions of things are connected through HID technology. We work with governments, educational institutions, hospitals, financial institutions, industrial businesses and some of the most innovative companies on the planet. Headquartered in Austin, Texas, HID Global has over 4,000 employees worldwide and operates international offices that support more than 100 countries. HID Global is an ASSA ABLOY Group brand.

For more information, visit www.hidglobal.com.